

An FPGA-based Accelerator for 3D CNNs Using FFT convolution

Abstract—Accelerating 3D CNNs poses a significant design challenge, due in large part to the increase in arithmetic complexity of the convolution operation in three dimensions. Winograd-based designs can achieve higher resource efficiency in networks using small kernels, yet are limited by increasingly complex transform operations in higher dimensions. FFT convolution has seen recent success in applying overlap-add to large CNNs. The Complex-domain multiplications required by the algorithm can likewise be optimized using polar coordinate representation. We propose a 3D CNN accelerator using FFT convolution augmented by overlap-add and CORDIC algorithms. We implement and test this accelerator on FPGA technology, achieving over double DSP-efficiency compared to the state-of-the-art.

Index Terms—Convolutional neural networks, Fast Fourier transforms, Computational efficiency, Algorithmic efficiency, Field programmable gate arrays, Accelerator architectures, Hardware acceleration.

1 INTRODUCTION

IMAGE processing demands have increased substantially in the last decade, due in part to the success of Convolutional Neural Networks (CNNs) in two-dimensional image recognition. Several accelerator architectures have been developed to attend to these demands; a method that has seen wide success in 2D [1] and 3D [2] CNNs.

A variety of optimization approaches have been developed for the task of accelerating CNNs, including: efficient memory access [3], [4], [5]; massive parallelization methods [6], [7] often via systolic-array inference [8]; optimizing floating-point operations [9]; quantizing weights and activations [10], [11]; and addressing data sparsity [4], [9].

One approach that has also garnered attention is the application of more efficient convolution algorithms. The standard discrete convolution algorithm, as well as the more widely used matrix-multiplication (MMX), have a base asymptotic complexity of $O(N^2)$, scaling very poorly with image and kernel sizes. This is particularly relevant to 3D CNNs, as the processing load of its convolution layers can quickly outscale all other layers in the network [12].

In 2016, Lavin & Gray [13] reviewed the relevant convolution algorithms — which were, at the time, MMX and FFT convolution — and proposed a new class of algorithms based on the Coppersmith–Winograd series of matrix multiplication methods [14]. MMX Algorithms with lower asymptotic bounds have been discovered [15], [16], however, they incur increasingly complex transforms which severely hinders their applicability. As such, the so-called Winograd class of convolution algorithms has become the standard for hardware-efficient CNN designs [17], [18], [19].

FFT convolution was deemed inefficient for modern CNN designs as its upsides only overweight its downsides for convolutions using large kernels [13], a method which has lost prevalence to deeper networks that use small (usually 3x3) kernels [20]. The algorithm is also not, as yet, easily implemented in high-level designs, due in large part to its reliance on Complex-domain operations.

Since then, new techniques were found to optimize key parts of the algorithm, such as convolution splitting [21],

overlap-add [22], [23], and mixed-radix FFT [24], as well as spectral CNNs [25], [26], [27], [28], [29]. It has also found success in CNNs required the use of large kernels, such as 3D CNNs [30], [31], [32].

There remains, however, the issue of Complex-domain multiplication. Recent FPGA implementations have addressed the issue using parallelization [33] and pipelining [9]. These methods, however, do not directly address the high operation count of Complex multiplication, instead deploying extra hardware resources to overcome it. As such, their hardware efficiency (measured in throughput per component) remains the same.

It is possible to reduce the operation count of Complex multiplication by representing its inputs as polar coordinates [34]. This was first implemented for CNNs in 2024 by Reis *et al.* [35] using high-level-synthesis (HLS), achieving promising results. However, HLS, while an important tool for quickly implementing designs, has been frequently criticized in CNN applications regarding resource underutilization [36], [37], [38], high power consumption [39] and general performance issues [40], [41], [42]. This is opposed to RTL models, written in hardware-description languages (HDL), where lower level optimizations are available, at the cost of longer development cycles.

We propose HECATE, an FFT-based 3D convolution accelerator using hardware-efficient polar-coordinate multiplication, an FFT pipeline, and optimized for large images using input slicing and overlap-add. We synthesize the architecture using HDL and implement it in a VC709 FPGA board. By validating our implementation using state-of-the-art benchmarks, we demonstrate the viability of FFT convolution in non-spectral 3D CNNs.

This document is organized as follows: In Section 2 we discuss the state-of-the art convolution algorithms and their computational complexity; In Section 3, we describe the design choices and optimizations of our proposed architecture; Section 4 details the procedures used to verify and test the architecture; Section 5 lists the results obtained and comparisons to the state-of-the-art; Section 6 discusses

related work; and Section 7 presents our conclusions.

2 CONVOLUTION ALGORITHMS

Comparative algorithmic analyses of Matrix Multiplication (MMX), Winograd, and FFT convolution have been performed previously by Lavin & Gray [13] as well as later publications [17]. Our focus in this Section is on an often overlooked property of these algorithms: their scalability; and how it affects the total complexity of the system.

Let us first define a standard nomenclature:

- I is a 3D input channel with dimensions $\{I_x, I_y, I_z\}$. There are I_c channels. Each of its elements are defined $I[ix, iy, iz, c]$.
- K is a 3D convolution kernel with dimensions $\{K_x, K_y, K_z\}$. There are K_f kernels. Each of its elements are defined $K[kx, ky, kz, f]$.
- Out is a 3D convolution output with dimensions $\{O_x, O_y, O_z\}$. There are I_c output channels and K_c filter maps. Each of its elements are defined $Out[ox, oy, oz, c, f]$
- $I[c] * K[f]$ is the convolution of one channel I and one kernel K .
- $I * K$ is the convolution of each channel I by each kernel K .

The dimension sizes $\{O_x, O_y, O_z\}$ are a function of the respective sizes of I and K and depends on the *mode* of convolution, as follows:

- *Full*: $O_d = I_d + K_d - 1$
- *Same*: $O_d = I_d$
- *Valid*: $O_d = I_d - K_d$

The baseline method of convolution used by CNNs, discrete convolution, uses *Same* mode and can be defined as

$$Out[ox, oy, oz, c, f] = \sum_{z=0}^{K_z} \sum_{y=0}^{K_y} \sum_{x=0}^{K_x} I[ox + kx, oy + ky, oz + kz, c] K[x, y, z, f]. \quad (1)$$

This method shares its complexity with the more commonly used matrix-multiplication algorithm [34]. To simplify calculations, we will consider the total number of multiplications required by each algorithm, here labeled M . For discrete and matrix-multiplication convolutions, this number is equal to the product of each dimension processed, i.e.,

$$M = I_x I_y I_z I_c K_x K_y K_z K_f. \quad (2)$$

2.1 FFT convolution

Lower complexity can be achieved via the Convolution Theorem:

$$I[c] * K[f] = \mathcal{F}^{-1}(\mathcal{F}(I[c]) \cdot \mathcal{F}(K[f])), \quad (3)$$

where $\cdot$ represents the pointwise product (also known as *Hadamard* product) and $\mathcal{F}$ and $\mathcal{F}^{-1}$ represent the Discrete Fourier Transform (DFT) and its inverse. This method calculates *full*-type convolution.

The class of algorithms known as *Fast Fourier Transforms* (FFT) are used to compute the DFTs. By definition, an FFT

computes an N -point DFT in $O(N \log_2 N)$. The standard radix-2 FFT [43], [44] breaks down the DFT into $\log_2 N$ steps, each consisting of multiple “butterfly operations” paired with twiddle-factor multiplications. The order of operations depend on the recursion method used: Decimation in Frequency (DIF) or Decimation in Time (DIT), as illustrated by Figures 1a and 1b respectively. Each “butterfly” receives a pair (A, B) and outputs $(A + B, A - B)$, where A and B are Complex numbers. Each twiddle-factor multiplier calculates a product by a root-of-unity ω with the form

$$\omega_N^k = e^{\frac{2\pi i}{N} k}, \quad (4)$$

where $k \in \mathbb{Z} \cap [0, N - 1]$.

While the input signal of the FFT is one-dimensional, it is possible to represent an arbitrary number of dimensions using zero-padding. Figure 2a represents a signal K given its dimensions all have size 2. To convolve this array with another signal of the same shape, we must first pad each dimension d by $2d - 1$, thus conveying its dimensionality and avoiding cyclic convolution. Figure 2b represents the padded signal; the gray line displays the order of rasterization.

The rasterized signal is further padded to its next power of two, such that radix-2 FFT can be applied across $\log_2 N$ stages. The size of the resulting array, denoted N , as a function of the sizes of K is

$$N = 2^{\lceil \log_2(2K_x - 1) \times \log_2(2K_y - 1) \times \log_2(2K_z - 1) \rceil}, \quad (5)$$

where $\lceil \cdot \rceil$ indicates the *ceiling* function (i.e., rounding up to the nearest integer).

The transformed array exiting the FFT chain is subject to *hermitian symmetry*. Assume an N -point array of Complex numbers of the form $(a + bi)$. If the array has hermitian symmetry, each point with index $(N/2) + 1$ to $N - 1$ is equivalent to the Complex conjugate (i.e., $(a - bi)$) of the points with index $(N/2) - 1$ to 1, respectively. This means that almost half of the array is redundant, and can be inferred by switching the sign bit of its y -component. This property is preserved after the pointwise multiplication, as such the total number of pointwise Complex multiplications required by FFT convolution is

$$MUL(\mathbb{C}) = \frac{N}{2} + 1. \quad (6)$$

In order to convolve I and K using FFT, the sizes of each transformed array must be equal. This can be achieved by zero-padding the smaller signal to match the larger one, however, this is inefficient in CNN applications where I is much larger than K . Instead, we can “slice” the input into kernel-sized pieces and convolve each slice with the kernel. The overlapping points of each convolution are added together, forming the complete output. This is known as *overlap-add* [23], [45] and is an example of algorithmic scaling.

Let S be a 3D slice of I over K , with dimensions $\{K_x, K_y, K_z\}$. The number of slices on each dimension is $\{S_x, S_y, S_z\}$, such that

$$S_x = \left\lceil \frac{I_x}{K_x} \right\rceil; S_y = \left\lceil \frac{I_y}{K_y} \right\rceil; S_z = \left\lceil \frac{I_z}{K_z} \right\rceil. \quad (7)$$

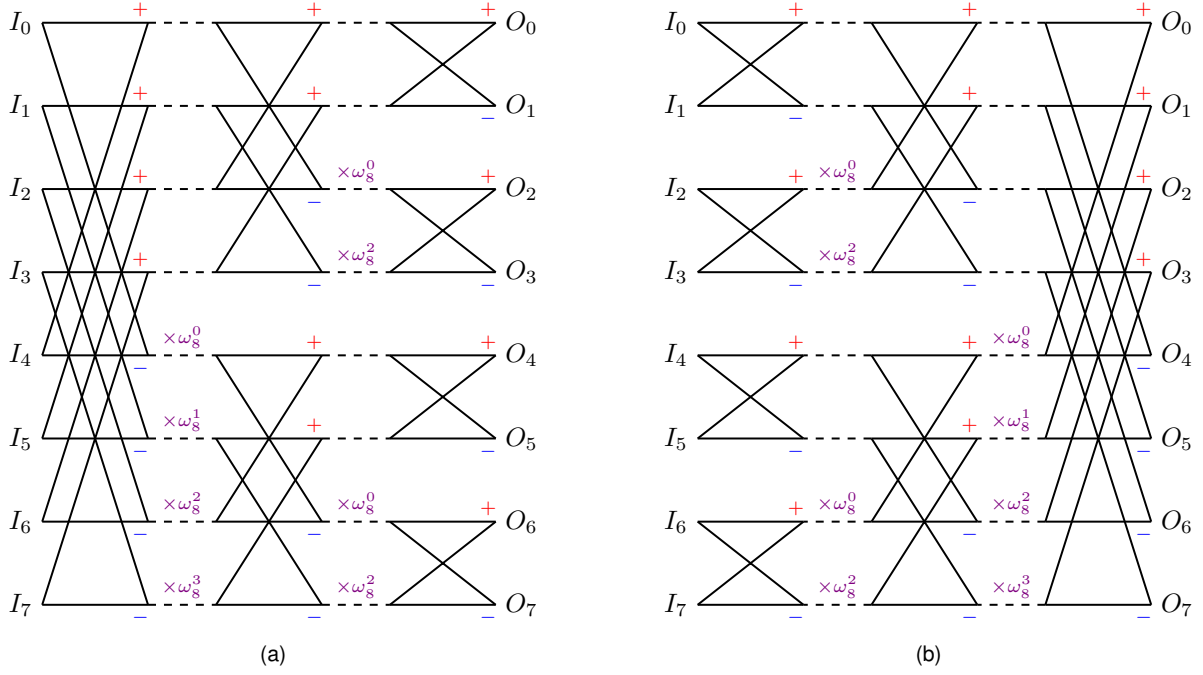


Fig. 1. Visual representation of an 8-point radix-2 FFT using Decimation in Frequency (a) and Decimation in Time (b).

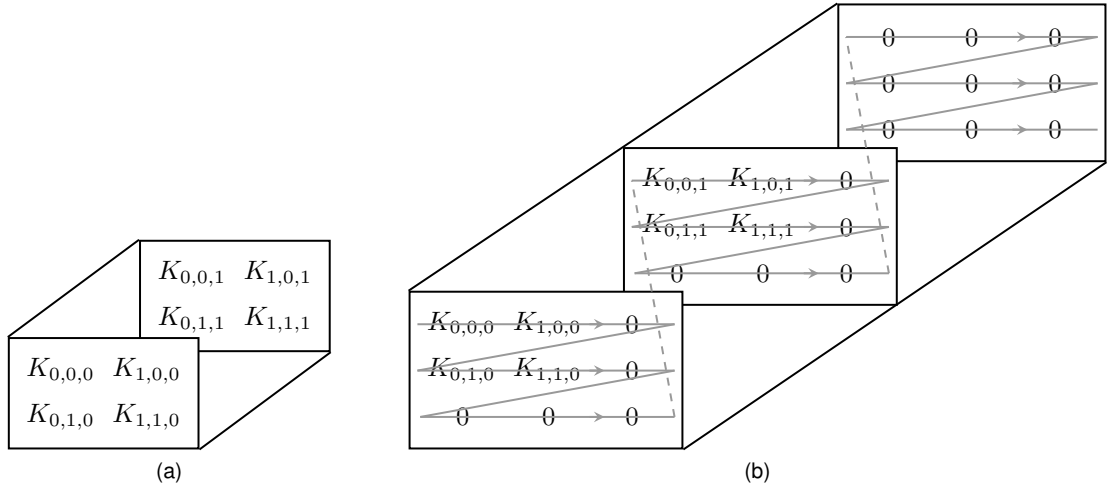


Fig. 2. A three-dimensional signal (a) and the required zero-padding to avoid cyclic convolution (b).

Complex-domain multiplication can be performed with 4 Real-domain multiplications. Let X and Y be Complex numbers such that $X = a + bi$ and $Y = c + di$. The product of the two signals is:

$$XY = (a + bi)(c + di) = (ac - bd) + (ad + bc)i. \quad (8)$$

As such, the total number of multiplications required by FFT convolution is

$$4 \times \left\lceil \frac{I_x}{K_x} \right\rceil \left\lceil \frac{I_y}{K_y} \right\rceil \left\lceil \frac{I_z}{K_z} \right\rceil \times \left(\frac{N}{2} + 1 \right). \quad (9)$$

However, X and Y can also be represented in polar coordinates such that $X = A/\alpha_1$ and $Y = B/\alpha_2$, where A, B are radial distances and α_1, α_2 are angular coordinates. As such

$$XY = A/\alpha_1 \cdot B/\alpha_2 = AB/\alpha_1 + \alpha_2. \quad (10)$$

This representation reduces the total operation count to 1 Real multiplication and 1 Real addition. Section 4 describes the algorithm used to implement the conversions between these representations. Notably, they do not incur any multiplications, bringing the total down to

$$\left\lceil \frac{I_x}{K_x} \right\rceil \left\lceil \frac{I_y}{K_y} \right\rceil \left\lceil \frac{I_z}{K_z} \right\rceil \times \left(\frac{N}{2} + 1 \right). \quad (11)$$

2.2 Winograd

Similarly to FFT convolution, the Winograd method transforms the input and kernel into a domain (more strictly, a *field*) where pointwise multiplication is equivalent to the convolution of the original signals [13]. The algorithm defines convolution as $F(m^3, r^3)$, where m and r are integers.

Let A and B be matrices such that their total size is equal to $(m + r - 1)^3$ and r^3 respectively. Their convolution is

$$A * B = (Z((XAX^T)^R X^T \cdot (YAY^T)^R Y^T)Z^T)^R Z^T,$$

where the superscript R denotes clockwise rotation and T denotes transposing. The result of this operation has size m^3 and constitutes a *valid*-type convolution.

The transform matrices depend on m and r , as well as the point-selection procedure used. 3D CNN applications require at least $F(2^3, 3^3)$ and trivial matrices for this operation have been found, such as the set used by Shen *et al.* (2018) [17]:

$$X = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}; Y = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix};$$

$$Z = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix}.$$

These matrices are considered *trivial* because their multiplication by another matrix is computationally efficient: multiplying by 0 or 1 is trivial and multiplying by $\frac{1}{2}$ can be computed as a right-bit-shift. Higher values of m and r incur different matrices where this property cannot be guaranteed.

Similarly to the FFT method, it is possible to calculate multiple $F(m^3, r^3)$ convolutions and add the overlapping points to form a complete output [31]. For an image I , the total number of multiplications is

$$\left\lceil \frac{I_x - r}{m} \right\rceil \left\lceil \frac{I_y - r}{m} \right\rceil \left\lceil \frac{I_z - r}{m} \right\rceil (m + r - 1)^3. \quad (12)$$

2.3 Comparison

Figures 3 and 4 are plots of Equations 2 (baseline), 12 (Winograd), 9 (FFT convolution) and 11 (FFT convolution using polar multiplication); for different kernel sizes and assuming K is equilateral and has size k . Figure 3 assumes $I = (k + 1)^3$, or one semi-convolution. Figure 4 assumes $I = 128 \times 171 \times 16$, the input size used by C3D [46].

These graphs do not account for the multiplications stemming from Winograd's transform at higher kernel-sizes, nor the twiddle-factor multiplications used by FFT convolution, as they are both multiplications by constants and can be better optimized in hardware.

3 ARCHITECTURE DESIGN

In order to convert the image and kernel to-and-from polar coordinates, we implement CORDIC (COordinate Rotation DIgital Computer) [47], a trigonometric algorithm that iteratively calculates the rotation of a given vector by decreasingly large angles. It has two modes of operation: rotation and vectorization; as well as three inputs, here labeled (x, y, z) . In vectorization mode, the rectangular coordinates of a Complex number are inputted as (x_0, y_0) , and after j iterations, the radial distance and angular coordinate are outputted as (x_j, z_j) , respectively. Similarly, in rotation

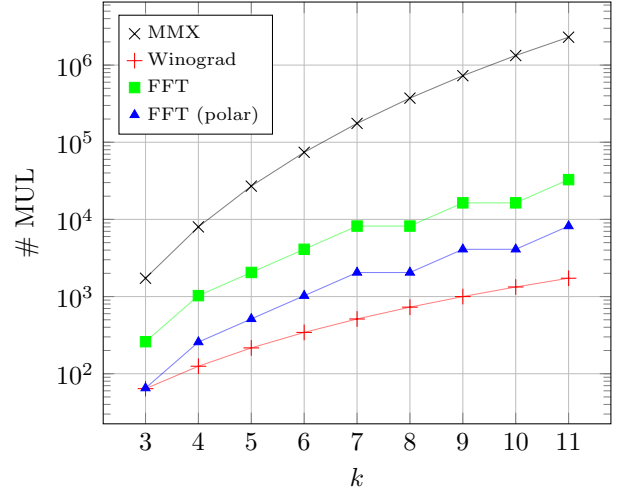


Fig. 3. Multiplications (log) vs kernel-size for $I = 4 \times 4 \times 4$

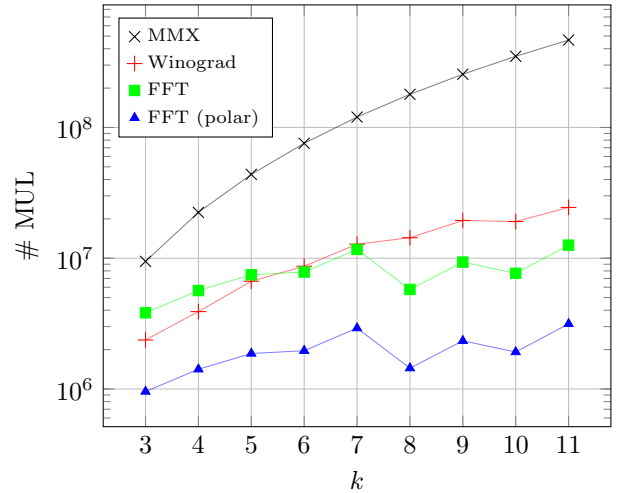


Fig. 4. Multiplications (log) vs kernel-size for $I = 128 \times 171 \times 16$

mode, the polar coordinates of a Complex number are inputted at (x_0, z_0) and after j iterations, the respective rectangular coordinates are outputted as (x_j, y_j) . This is generally expressed as

$$\begin{bmatrix} x_{j+1} \\ y_{j+1} \\ z_{j+1} \end{bmatrix} = \begin{bmatrix} x_j - \sigma_j 2^{-j} y_j \\ y_j + \sigma_j 2^{-j} x_j \\ z_j + \sigma_j \arctan(2^{-j}) \end{bmatrix}, \quad (13)$$

where

$$\sigma_j = \begin{cases} \text{sign}(y_j); & \text{in vectorization mode} \\ -\text{sign}(z_j); & \text{in rotation mode} \end{cases}. \quad (14)$$

The precision of the algorithm is equivalent to 2^{-j} , or 1-bit of precision per iteration. Note that when using fixed-point notation, it is enough to iterate once for each bit after the point. The total number of bits of a variable is denoted S_t , and the number of bits after the point is denoted S_p . Our implementation defaults to 24 and 16, respectively.

A simple hardware implementation of CORDIC consists of two S_t -bit adders, one S_p -bit adder, two variable

bit-shifters, three registers, and a lookup table for storing $\arctan(2^{-j})$.

The outputs of x_j and y_j are scaled by K_j on each iteration, where

$$K_j = \cos(\arctan(2^{-j})), \quad (15)$$

and must be corrected to obtain the desired result. The correction can be made after the process completes, by dividing each output by the scaling factor

$$\frac{1}{K} = \prod_{j=0}^{S_p-1} \frac{1}{\sqrt{1+2^{-2j}}} \stackrel{S_p=16}{\approx} 1.64676. \quad (16)$$

In the pointwise multiplication step, CORDIC is used 3 times. The first two convert the input image slice and kernel to polar coordinates, and the last converts the result back to rectangular coordinates. The scaling factor carries over across operations, including pointwise multiplication, and as such we can apply the correction at any point in the process (image, kernel, or product). The combined scaling factor is

$$\frac{1}{K^3} \stackrel{S_p=16}{\approx} 0.22393. \quad (17)$$

The values of $\arctan(2^{-j})$ are also constant and can be pre-calculated and stored in a lookup table. However, the convolution operation requires adding two angular coordinates before the transformation back into rectangular coordinates. The resulting angle can exceed π radians, causing issues with the algorithm. To circumvent this, we store the *quadrant* of each angular coordinate in the unit circle, normalize them to the first quadrant, perform the add operation, and convert the result into the respective quadrant. To simplify this computation, we divide each $\arctan(2^{-j})$ by 2π . The resulting binary, encoded in 16-bits, represents fractions of π , with its two most-significant bits representing the quadrant of the vector in the unit circle, as shown in Figure 5. This allows quick access to the quadrant of the angle, as well as enabling normalization to any quadrant by simply setting its first two bits.

CORDIC's complexity does not increase with image- or kernel-size, depending instead on S_p , giving it a relative asymptotic complexity of $O(1)$ and enabling a trade-off between numerical precision and operation complexity.

CORDIC can also be applied to FFTs to more efficiently calculate twiddle-factor multiplications [48], [49]. Each CORDIC iteration rotates the input by elementary angles which coincide with the twiddle-factors used by the FFT. This, however, requires scaling each point that wasn't multiplied by the twiddle-factor, to account for the gain induced by CORDIC, resulting in an increase of LUT components used. As later described in Section 4, LUTs are the limiting factor in scaling this design, not DSPs. As such, we elected to implement the twiddle-factor rotations using DSP blocks.

Our twiddle-factor multiplication design using DSPs is a variant of the Karatsuba multiplication algorithm [50]. Using the previously established nomenclature for Complex numbers, Karatsuba defines three intermediate variables

$$\begin{cases} k1 = c \times (a + b) \\ k2 = b \times (c + d) \\ k3 = a \times (d - c) \end{cases}, \quad (18)$$

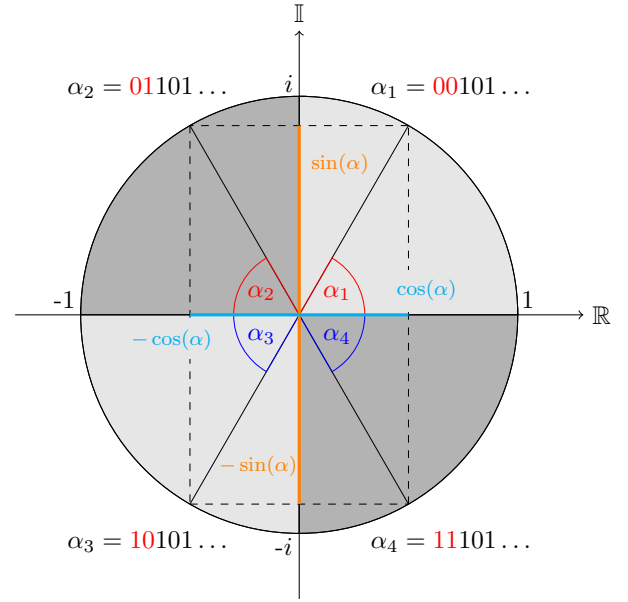


Fig. 5. Four symmetric angles encoded as π -fractions.

such that

$$XY = (k1 - k2) + (k1 + k3)i. \quad (19)$$

As such, the algorithm normally incurs 5 additions and 3 multiplications. However, in the case of twiddle-factor multiplication, c and d are constant. By extension, $(c + d)$ and $(d - c)$ are also constant, and can be stored in LUT for use in the algorithm. This lowers the operation count to 3 adds and 3 muls, a substantial improvement over the standard 2 adds and 4 muls of Equation 8.

3.1 Pipelining

Each step of the proposed algorithm can be pipelined to greatly increase its throughput. Figure 6 illustrates our pipelined design, optimized for hardware efficiency. The system consists of two phases, as delimited by the gray line. Hardware across phases is reused instead of replicated.

In Phase-I, the 3D slice is rasterized and padded as shown in Figure 2, then multiplexed into the FFT chain.

The FFT is divided into $\log_2 N$ stages, where N is defined by Equation 5. Each stage consists of one butterfly, one twiddle-factor multiplication block, and a set of registers. The data entering the first FFT stage is still on the Real domain. As such, we arranged this FFT in decimation-in-frequency (see figure 1a), as this mode places most non-redundant twiddle-factor multiplications at the first stage, significantly simplifying these operations.

The add and subtract blocks within the butterflies are (on every layer except the first) Complex operation blocks, and as such each constitute two operations each. The twiddle-factor multiplication blocks past the first stage were implemented using Karatsuba, resulting in 3 adders and 3 multipliers each, along with a lookup table that is shared across stages. In reality, many of these operations will be optimized away by the synthesizer, as about half the input array is zeroes. Some twiddle-factors are also trivial to multiply by, such as $\omega^0 = 1$ and $\omega^{\frac{N}{4}} = i$.

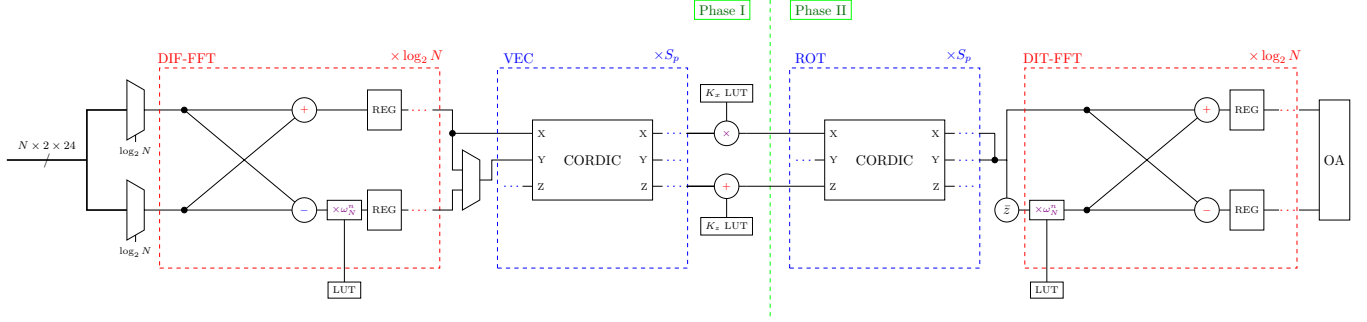


Fig. 6. Fully-pipelined FFT-conv design.

Each FFT node processes two points of the input per machine cycle, storing the result into the register bank. On the next cycle, the node will process two more points, while the proceeding stage will read the result and repeat the process along the chain. Once the pipeline is full, the FFT chain is capable of processing two points of input per cycle.

Hermitian symmetry carries over through CORDIC and polar multiplication. As such, the points past $N/2$ can be inferred much later. The edge case for $N/2$ still needs to be calculated, as it is not the Complex conjugate of index 0. Its insertion into the next block is done via a multiplexer.

The CORDIC chain is set to vectorization on Phase-I, to perform the conversion to polar coordinates. Similarly to the FFT chain, there are S_p blocks which, when fully pipelined, process two points (one symmetrical) of the original array.

The polar multiplication step consists of a one multiplier and one adder, in accordance to Equation 10. The kernels are processed at synthesis time and their Complex polar coordinates are stored in their respective lookup tables. Before being stored, each kernel's radial distance is multiplied by the CORDIC correction factor (see Equation 17), thus applying the correction to each point of the array without the need for additional multiplier blocks.

In Phase-I, each slice S of each channel I_c is processed up to the multiplication by K and stored in a register bank. After the process is complete, Phase-II begins, where the CORDIC chain is set to rotation mode and the FFT is rearranged into DIT mode. This process is managed by a control unit and routing elements, not depicted in the Figure to avoid clutter.

The output at CORDIC during Phase-II is joined at the final stage, forming one point of the N -sized array. The respective Complex conjugate (denoted $\bar{z}$) is inferred and sent into the FFT chain, now in DIT mode. The $N/2$ exception to hermitian symmetry is always a Real number. As such, polar coordinate conversion is trivial and need not be processed by CORDIC.

The FFT chain now executes the inverse transform. The last stage of DIF is equivalent to the first stage of DIT (see Figure 1) allowing data to flow seemingly along the pipeline. As the mirror to DIF, DIT places most non-redundant twiddle-factor multiplications at the end. The data leaving the system is on the Real domain, as all Complex parts are guaranteed to cancel-out, thus simplifying these operations as well.

Finally, the overlap-add module (OA) receives the two Real points and adds them to the respective indexes in the

feature map. This process is repeated for all channels and all kernels. As such, Phase-II is K_f times longer than Phase-I.

3.2 Parallelism

The design in Figure 6 is compact and scalable, useful for implementation across a range of FPGA devices. However, we can further improve the hardware efficiency of the design using parallelism.

Previously, we only synthesized one butterfly and twiddle-factor multiplier per FFT stage. We can instead synthesize every block on each stage, allowing the synthesizer to better optimize the redundant twiddle-factor multiplications.

To maintain uninterrupted data flow along the pipeline, the CORDIC chain is scaled accordingly. Similarly to the FFT chain, many CORDIC blocks will become redundant and will be optimized away during synthesis.

The polar multiplication and overlap-add stages are similarly parallelized, completing one slice per cycle in Phase-II.

This version of the architecture is labeled **parallel-core**, while the design described in Section 3.1 is accordingly labeled **pipelined-core**.

4 IMPLEMENTATION & TESTING

We synthesized and implemented both designs on a VC709 FPGA board. Timing closure was achieved at 150 MHz for parallel-core and 200 MHz for pipelined-core, due to routing and congestion factors.

Software testing was carried out to ascertain correctness and measure precision. A randomized trial was performed, comparing its results to a standard convolution algorithm at each iteration. Edge-case trials were also performed to ascertain correctness for very large or small inputs.

Table 1 displays the resource usage metrics of both cores for $K = 3 \times 3 \times 3$.

TABLE 1
Resource usage of pipelined and parallel cores in VC709 environment.

Core	DSP	LUT	FF
Pipelined	1	5861	4265
[usage]	0.03%	1.3%	0.5%
Parallel	65	296572	213376
[usage]	1.8%	66.9%	25.6%

TABLE 2
Comparison of different FPGA accelerator architectures applied on C3D.

Project	Shen <i>et al.</i> (2018) [17]		Liu <i>et al.</i> (2019) [56]	Fan <i>et al.</i> (2021) [9]	Yang <i>et al.</i> (2023) [57]	Our Design
Algorithm	Winograd		MMX	MMX	MMX	FFT
Precision	16-fixed		16-fixed	<16-float	(unmentioned)	16-fixed
Board	VC709	VUS440	VC709	GX1150	XCZU15	VC709
Technology (nm)	28	20	28	20	16	28
Frequency (MHz)	150	200	120	220	100	150
DSP	1536	1536	3595	1345	1317	65
Throughput (GOP/s)	474.3	940.6	667.7	1536	209.4	107.8
Resource efficiency (20)	0.31	0.61	0.19	1.14	0.16	1.66
Normalized (21)	2.06	3.06	1.55	5.19	1.59	11.05

Throughput (GOP/s) was measured on the parallel-core using the first CONV layer of C3D [46], where $I = 128 \times 171 \times 3$, $I_c = 3$ and $K_f = 16$.

Resource efficiency was calculated as the product of throughput and number of DSPs inferred [9], [37], [51], [52]. This is expressed as

$$\frac{GOP}{s \times DSP}. \quad (20)$$

This metric can be normalized across different FPGA devices by dividing it by the operational frequency [37], [53], [54]. This is expressed as

$$\frac{GOP}{s \times DSP \times MHz}. \quad (21)$$

5 RESULTS

Software testing using 24-bit fixed-point values with 16 fractional bits revealed a worst-case precision of 2^{-8} . In other words, the 16 most significant bits always returned the correct value across both the randomized and edge-case trials. This is sufficient for both training and inference of CNNs [55].

To the extent of our research regarding FPGA-based acceleration of 3D CNNs, Fan *et al.* [9] achieves the highest resource efficiency. Their choice of convolution algorithm is the standard matrix-multiplication, however, the implementation of floating-point quantization enables more efficient operations within the algorithm. Shen *et al.* (2018) [17] remains the most resource efficient FPGA-based accelerator that uses Winograd convolution and that is capable of processing 3D convolutional filters.

Table 2 displays the throughput (GOP/s), hardware efficiency (20), and normalized hardware efficiency (21) of our project as well as the state-of-the-art. This comparison is analogous to the ones made by previous projects in their analyses [9], [17]. Liu *et al.* [56] and Yang *et al.* [57] have also made contributions to the state-of-the-art and were included in this comparison.

6 RELATED WORK

CNN acceleration is a broad and actively-researched field. This section covers some relevant bibliography not previously mentioned.

Both abstract network architecture to hardware-implemented accelerators can be used in tandem to achieve higher performance and efficiency. Spectral CNNs [25], [26], [27], [28], [29] as well as rotation-invariant CNNs [58], [59] avoid the need to transform to-and-from the Complex domain using FFTs. This incurs changes across each layer of the network, and more research is needed to evaluate whether the impact on accuracy and development cycles are worth the performance gain.

Since Shen *et al.* (2018) [17], algorithmic optimizations of the Winograd method have been published [18], [60], [61] but have not yet been implemented in an FPGA-based accelerator. There exist more recent applications of the algorithm for CNNs [37], [62] but they are not trivially expandable to three-dimensions.

Winograd and FFT Convolutions may be able to benefit from similar quantization methods as the ones used by Fan *et al.* [9]. Their implementation does not optimize the total number of operations of the algorithm, instead optimizing the operations themselves.

The possibility of designing CNNs using CORDIC activation functions have been proposed [63], [64]. This would likely lead to more optimal accelerators using FFT convolution.

There exist other FFT algorithms such as split-radix [65] and prime-factor [66] FFTs, and might be useful as a way to optimize slice dimensions.

7 CONCLUSIONS & FUTURE WORK

We demonstrate the lower complexity bounds of FFT convolution, derived from its efficient scaling algorithm, in comparison to related algorithms. We design and implement an FPGA-based accelerator using polar-coordinate multiplication, achieving state-of-the-art resource efficiency.

Our parallel-core design, while highly DSP-efficient, incurs significant LUT and FF usage. These values stem from the twiddle-factor multiplications on our accelerator's FFT pipeline. The literature reveals a trend of new optimizations for FFT and CORDIC in the FPGA design space, enabling future optimizations of the design. Applications in more recent FPGA devices will also likely yield further optimization.

Based on the data presented in this work, we suggest that FFT convolution, paired with overlap-add and polar

multiplication, is the superior method for efficiently accelerating 3D CNNs, as well as other networks reliant on large kernels.

REFERENCES

- [1] S. Mittal, "A survey of fpga-based accelerators for convolutional neural networks," *Neural computing and applications*, vol. 32, no. 4, pp. 1109–1139, 2020.
- [2] S. Mittal and Vibhu, "A survey of accelerator architectures for 3d convolution neural networks," *Journal of Systems Architecture*, vol. 115, p. 102041, 2021.
- [3] X. Han, D. Zhou, S. Wang, and S. Kimura, "Cnn-merp: An fpga-based memory-efficient reconfigurable processor for forward and backward propagation of convolutional neural networks," in *2016 IEEE 34th International Conference on Computer Design (ICCD)*. IEEE, 2016, pp. 320–327.
- [4] C. Jiang, D. Ojika, B. Patel, and H. Lam, "Optimized fpga-based deep learning accelerator for sparse cnn using high bandwidth memory," in *IEEE Annual International Symposium on Field-Programmable Custom Computing Machines*, 2021.
- [5] M. Doumet, M. Stan, M. Hall, and V. Betz, "H2pipe: high throughput cnn inference on fpgas with high-bandwidth memory," in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2024, pp. 69–77.
- [6] N. P. Jouppi, D. H. Yoon, G. Kurian, S. Li, N. Patil, J. Laudon, C. Young, and D. Patterson, "A domain-specific supercomputer for training deep neural networks," *Communications of the ACM*, vol. 63, no. 7, pp. 67–78, 2020.
- [7] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "Nvidia a100 tensor core gpu: Performance and innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021.
- [8] S. H. Chua, T. H. Teo, M. A. Tiruye, and I.-C. Wey, "Systolic array based convolutional neural network inference on fpga," in *2022 IEEE 15th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*. IEEE, 2022, pp. 128–133.
- [9] H. Fan, S. Liu, Z. Que, X. Niu, and W. Luk, "High-performance acceleration of 2-d and 3-d cnns on fpgas using static block floating point," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 4473–4487, 2021.
- [10] Y. He, W. Liu, M. Tahir, Z. Li, S. Zhang, and H. B. Amur, "Research on efficient cnn acceleration through mixed precision quantization: A comprehensive methodology," *International Journal of Advanced Computer Science & Applications*, vol. 14, no. 12, 2023.
- [11] O. B. H. Salah, S. Messaoud, M. A. Hajjaji, M. Atri, and N. Liouane, "Post-training quantization for efficient fpga-based neural network acceleration," *Integration*, p. 102508, 2025.
- [12] K. Hegde, R. Agrawal, Y. Yao, and C. W. Fletcher, "Morph: Flexible acceleration for 3d cnn-based video understanding," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 933–946.
- [13] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 4013–4021.
- [14] D. Coppersmith and S. Winograd, "On the asymptotic complexity of matrix multiplication," *SIAM Journal on Computing*, vol. 11, no. 3, pp. 472–492, 1982.
- [15] E. Karstadt and O. Schwartz, "Matrix multiplication, a little faster," *Journal of the ACM (JACM)*, vol. 67, no. 1, pp. 1–31, 2020.
- [16] O. Schwartz and N. Vaknin, "Pebbling game and alternative basis for high performance matrix multiplication," *SIAM Journal on Scientific Computing*, vol. 45, no. 6, pp. C277–C303, 2023.
- [17] J. Shen, Y. Qiao, Y. Huang, M. Wen, and C. Zhang, "Towards a multi-array architecture for accelerating large-scale matrix multiplication on fpgas," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2018, pp. 1–5.
- [18] B. Barabasz and D. Gregg, "Winograd convolution for dnns: Beyond linear polynomials," in *AI* IA 2019—Advances in Artificial Intelligence: XVIIIth International Conference of the Italian Association for Artificial Intelligence, Rende, Italy, November 19–22, 2019, Proceedings*. Springer, 2019, pp. 307–320.
- [19] M. Vardhana and R. Pinto, "High-performance winograd based accelerator architecture for convolutional neural network," *IEEE Computer Architecture Letters*, 2025.
- [20] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [21] K. Chitsaz, M. Hajabdollahi, N. Karimi, S. Samavi, and S. Shirani, "Acceleration of convolutional neural network using fft-based split convolutions," *arXiv preprint arXiv:2003.12621*, 2020.
- [22] T. Highlander and A. Rodriguez, "Very efficient training of convolutional neural networks using fast fourier transform and overlap-and-add," *arXiv preprint arXiv:1601.06815*, 2016.
- [23] Y. Chen, L. De-Jian, F. Xi, C. Chong-Fei, W. Ji-Ceng, and L. Yu, "Fast fourier transform convolutional neural network accelerator based on overlap addition," *Journal of China Universities of Posts and Telecommunications*, vol. 31, no. 5, p. 71, 2024.
- [24] Y. Meng, J. Wu, S. Xiang, J. Wang, J. Hou, Z. Lin, and C. Yang, "A high-throughput and flexible cnn accelerator based on mixed-radix fft method," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.
- [25] H. Pratt, B. Williams, F. Coenen, and Y. Zheng, "Fcnn: Fourier convolutional neural networks," in *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2017, Skopje, Macedonia, September 18–22, 2017, Proceedings, Part I* 17. Springer, 2017, pp. 786–798.
- [26] S. Liu, H. Fan, and W. Luk, "Accelerating fully spectral cnns with adaptive activation functions on fpga," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1530–1535.
- [27] S. O. Ayat, M. Khalil-Hani, A. A.-H. Ab Rahman, and H. Abdellatif, "Spectral-based convolutional neural network without multiple spatial-frequency domain switchings," *Neurocomputing*, vol. 364, pp. 152–167, 2019.
- [28] Y. Han and B.-W. Hong, "Deep learning based on fourier convolutional neural network incorporating random kernels," *Electronics*, vol. 10, no. 16, p. 2004, 2021.
- [29] T. Watanabe and D. F. Wolf, "Image classification in frequency domain with 2srelu: a second harmonics superposition activation function," *Applied Soft Computing*, vol. 112, p. 107851, 2021.
- [30] N. Nguyen-Thanh, H. Le-Duc, D.-T. Ta, and V.-T. Nguyen, "Energy efficient techniques using fft for deep convolutional neural networks," in *2016 International Conference on Advanced Technologies for Communications (ATC)*. IEEE, 2016, pp. 231–236.
- [31] Q. Lan, Z. Wang, M. Wen, C. Zhang, and Y. Wang, "High performance implementation of 3d convolutional neural networks on a gpu," *Computational intelligence and neuroscience*, vol. 2017, no. 1, p. 8348671, 2017.
- [32] X. Li, Y. Chu, and X. Wu, "3d convolutional neural network based on spatial-spectral feature pictures learning for decoding motor imagery eeg signal," *Frontiers in Neuroinformatics*, vol. 18, p. 1485640, 2024.
- [33] X. Li, Y. He, W. Zhu, W. Qu, Y. Li, C. Li, and B. Zhu, "Split_composite: A radar target recognition method on fft convolution acceleration," *Sensors*, vol. 24, no. 14, p. 4476, 2024.
- [34] R. C. Lera and B. C. Albertini, "Hardware-efficient convolution algorithms for cnn accelerators: A brief review," in *Anais do XX Encontro Nacional de Inteligência Artificial e Computacional*. SBC, 2023, pp. 86–96.
- [35] E. Reis, T. Akilan, and M. Khalid, "Phasor-driven acceleration for fft-based cnns," *arXiv preprint arXiv:2406.00290*, 2024.
- [36] Y. Xu, J. Luo, and W. Sun, "Flare: An fpga-based full precision low power cnn accelerator with reconfigurable structure," *Sensors*, vol. 24, no. 7, p. 2239, 2024.
- [37] J. Jiang, Y. Zhou, Y. Gong, H. Yuan, and S. Liu, "Fpga-based acceleration for convolutional neural networks: A comprehensive review," *arXiv preprint arXiv:2505.13461*, 2025.
- [38] P. Antunes and A. Podobas, "Fpga-based neural network accelerators for space applications: A survey," *arXiv preprint arXiv:2504.16173*, 2025.
- [39] A. M. Salman, A. S. Tulan, R. Y. Mohamed, M. H. Zakhari, and H. Mostafa, "Comparative study of hardware accelerated convolution neural network on pynq board," in *2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*. IEEE, 2020, pp. 578–582.
- [40] C. J. Faber, S. D. Harris, Z. Xiac, R. D. Chamberlain, and A. M. Cabrera, "Challenges designing for fpgas using high-level synthesis," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2022, pp. 1–7.
- [41] M. R. Ahmed, T. Koike-Akino, K. Parsons, and Y. Wang, "Autohls: Learning to accelerate design space exploration for hls designs," in *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, 2023, pp. 491–495.

- [42] J. Lau, Y. Xiao, Y. Xie, Y. Chi, L. Song, S. Xiang, M. Lo, Z. Zhang, J. Cong, and L. Guo, "Rapidstream ir: Infrastructure for fpga high-level physical synthesis," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–11.
- [43] C. F. Gauss, "Nachlass: Theoria interpolationis methodo nova tractata," *Carl Friedrich Gauss Werke*, vol. 3, pp. 265–327, 1866.
- [44] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [45] S. W. Smith, *Convolution*. San Diego, CA, USA: California Technical Publishing, 1997, ch. 6, pp. 107–122.
- [46] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3d convolutional networks," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 4489–4497.
- [47] J. Volder, "The cordic computing technique," in *Papers presented at the the March 3-5, 1959, western joint computer conference*, 1959, pp. 257–261.
- [48] A. Banerjee, A. S. Dhar, and S. Banerjee, "Fpga realization of a cordic based fft processor for biomedical signal processing," *Microprocessors and Microsystems*, vol. 25, no. 3, pp. 131–142, 2001.
- [49] P. Choudhary and A. Karmakar, "Cordic based implementation of fast fourier transform," in *2011 2nd International Conference on Computer and Communication Technology (ICCCCT-2011)*. IEEE, 2011, pp. 550–555.
- [50] A. A. Karatsuba and Y. P. Ofman, "Multiplication of many-digit numbers by automatic computers," in *Doklady Akademii Nauk*, vol. 145, no. 2. Russian Academy of Sciences, 1962, pp. 293–294.
- [51] X. Li, H. Huang, T. Chen, H. Gao, X. Hu, and X. Xiong, "A hardware-efficient computing engine for fpga-based deep convolutional neural network accelerator," *Microelectronics Journal*, vol. 128, p. 105547, 2022.
- [52] A. Kyriakos, E.-A. Papatheofanous, C. Bezaitis, and D. Reisis, "Resources and power efficient fpga accelerators for real-time image classification," *Journal of Imaging*, vol. 8, no. 4, p. 114, 2022.
- [53] M. Meyer, T. Kenter, and C. Plessl, "Evaluating fpga accelerator performance with a parameterized opencl adaptation of the hpc-challenge benchmark suite," *arXiv preprint arXiv:2004.11059*, 2020.
- [54] A. Montgomerie-Corcoran, Z. Yu, J. Cheng, and C.-S. Bouganis, "Pass: Exploiting post-activation sparsity in streaming architectures for cnn acceleration," in *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2023, pp. 288–293.
- [55] M. Courbariaux, Y. Bengio, and J.-P. David, "Training deep neural networks with low precision multiplications," *arXiv preprint arXiv:1412.7024*, 2014.
- [56] Z. Liu, P. Chow, J. Xu, J. Jiang, Y. Dou, and J. Zhou, "A uniform architecture design for accelerating 2d and 3d cnns on fpgas," *Electronics*, vol. 8, no. 1, p. 65, 2019.
- [57] Y. Yang, W. He, J. Hu, S. Cheng, W. Xu *et al.*, "Fgpa-based design for accelerating 3d convolutional neural network," *International Journal of Frontiers in Engineering Technology*, vol. 5, no. 3, pp. 40–48, 2023.
- [58] J. Kim, W. Jung, H. Kim, and J. Lee, "Cycnn: A rotation invariant cnn using polar mapping and cylindrical convolution layers," *arXiv preprint arXiv:2007.10588*, 2020.
- [59] H. Mo and G. Zhao, "Ric-cnn: rotation-invariant coordinate convolutional neural network," *Pattern Recognition*, vol. 146, p. 109994, 2024.
- [60] B. Barabasz, A. Anderson, K. M. Soodhalter, and D. Gregg, "Error analysis and improving the accuracy of winograd convolution for deep neural networks," *ACM Transactions on Mathematical Software (TOMS)*, vol. 46, no. 4, pp. 1–33, 2020.
- [61] S. A. Alam, A. Anderson, B. Barabasz, and D. Gregg, "Winograd convolution for deep neural networks: Efficient point selection," *ACM Transactions on Embedded Computing Systems*, vol. 21, no. 6, pp. 1–28, 2022.
- [62] S. Kala, B. R. Jose, J. Mathew, and S. Nalesh, "High-performance cnn accelerator on fpga using unified winograd-gemm architecture," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 12, pp. 2816–2828, 2019.
- [63] W. Shen, J. Jiang, M. Li, and S. Liu, "Efficient cordic-based activation functions for rnn acceleration on fpgas," *IEEE Transactions on Artificial Intelligence*, 2024.
- [64] O. Kokane, G. Raut, S. Ullah, M. Lokhande, A. Teman, A. Kumar, and S. K. Vishvakarma, "Retrospective: A cordic based configurable activation function for nn applications," *arXiv preprint arXiv:2503.14354*, 2025.
- [65] H. Sorensen, M. Heideman, and C. Burrus, "On computing the split-radix fft," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 34, no. 1, pp. 152–156, 2003.
- [66] C. Burrus and P. Eschenbacher, "An in-place, in-order prime factor fft algorithm," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 29, no. 4, pp. 806–817, 1981.